



Rapport de stage d'été

Année universitaire 2024–2025

Conception et développement d'une plateforme intelligente d'assistance à l'ingénierie de détection basée sur la Cyber Threat Intelligence et l'intelligence artificielle

Entreprise d'accueil : Forvis Mazars Group

Durée du stage : 1 mois

Stagiaire : Ilyes

Nature du travail : Projet de stage et démonstrateur technique

Rapport de stage d'été

2024–2025

Remerciements

Je tiens à remercier Forvis Mazars Group pour l'accueil qui m'a été réservé durant ce stage d'un mois et pour le cadre professionnel qui m'a permis de travailler sur un sujet concret lié à la cybersécurité opérationnelle.

Je remercie également les personnes qui m'ont accompagné, orienté et challengé pendant la réalisation du projet. Leurs remarques m'ont aidé à dépasser une approche purement fonctionnelle pour réfléchir à la fiabilité, à la traçabilité et aux limites d'un système utilisant l'intelligence artificielle dans un contexte de sécurité.

Enfin, je remercie mon établissement universitaire pour l'encadrement pédagogique et pour l'opportunité de formaliser ce travail sous la forme d'un rapport de stage structuré.

Résumé

Ce rapport présente la conception et le développement d'une plateforme intelligente d'assistance à l'ingénierie de détection. Le travail réalisé vise à transformer des renseignements de Cyber Threat Intelligence issus de MISP en propositions de règles Sigma, tout en conservant des mécanismes de contrôle déterministes et une validation humaine. La plateforme s'appuie sur FastAPI, PostgreSQL, Redis, Celery, LangGraph, Next.js, MISP, Sigma et pySigma. L'intelligence artificielle intervient pour extraire des comportements, proposer des mappings MITRE ATT&CK et générer ou réparer des candidats de détection. Elle reste cependant encadrée par des watchers, des scores de confiance et de trust déterministe, une validation pySigma, une détection de doublons et une file de revue analyste.

Le travail réalisé correspond à un projet de stage et non à un produit commercial prêt pour une production d'entreprise. Les principaux résultats sont une architecture Docker locale, une ingestion MISP, une orchestration de workflow persistée, une interface SOC et une documentation technique. Les limites identifiées concernent notamment le durcissement de l'authentification, l'observabilité, les tests frontend, la vérification complète du mode IA réel et l'intégration SIEM finale.

Mots-clés : CTI, MISP, MITRE ATT&CK, Sigma, pySigma, LangGraph, FastAPI, Next.js, détection, SOC, intelligence artificielle.

Abstract

This internship report presents the design and implementation of an AI-assisted detection engineering platform. The platform consumes Cyber Threat Intelligence from MISP and transforms it into reviewed Sigma detection proposals. The AI component supports behavior extraction, ATT&CK mapping, Sigma generation and repair, but deterministic gates and human analyst approval remain mandatory before catalog publication. The system uses FastAPI, PostgreSQL, Redis, Celery, LangGraph, Next.js, MISP, Sigma and pySigma.

The project demonstrates a realistic local architecture for CTI ingestion, workflow persistence, watcher-based control, confidence and trust scoring, duplicate detection and analyst review. It is not presented as a production-ready commercial system. Remaining limitations include enterprise authentication, monitoring, frontend test automation, full live AI validation and real SIEM deployment.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Introduction générale	1
1 Cadre du stage et contexte professionnel	2
1.1 Présentation de Forvis Mazars Group	2
1.2 Contexte du stage	3
1.3 Sujet et problématique	3
1.4 Objectifs poursuivis	4
1.5 Organisation du rapport	4
2 État de l’art et fondements scientifiques	5
2.1 Cyber Threat Intelligence et exploitation opérationnelle	5
2.2 SOC, SIEM et ingénierie de détection	5
2.3 Threat Hunting et approche comportementale	6
2.4 Sigma et validation indépendante du SIEM	6
2.5 IA humaine dans la boucle	7
2.6 IA explicable, confiance et gouvernance	7
2.7 Cadres de référence complémentaires	7
3 Analyse des besoins et cahier des charges	9

3.1	Expression du besoin	9
3.2	Besoins fonctionnels	9
3.3	Besoins non fonctionnels	10
3.4	Contraintes de sécurité	11
3.5	Cas d'utilisation	11
3.6	Cycle de vie attendu d'une détection	11
3.7	Critères d'acceptation	12
4	Conception et architecture de la solution	13
4.1	Principes de conception	13
4.2	Architecture générale	13
4.3	Architecture Docker	14
4.4	Conception du workflow IA	15
4.5	Ingestion MISP	15
4.6	Modèle de données	15
4.7	Rôle des watchers et des scores	16
5	Réalisation de la plateforme	17
5.1	Réalisation du backend	17
5.2	Persistance et migrations	18
5.3	Ingestion MISP et consommation	18
5.4	Orchestration asynchrone	19
5.5	Réalisation du moteur de raisonnement	19
5.6	Génération et validation Sigma	19
5.7	Interface SOC	20
5.8	Cycle de revue et publication	23
6	Validation, résultats et limites	24
6.1	Méthode de validation	24
6.2	Scénarios fonctionnels vérifiés	25
6.3	Résultats obtenus	25

6.4	Difficultés rencontrées	25
6.5	Limites identifiées	26
6.6	Bilan	26
Conclusion générale		27
Bibliographie		28
A Annexe A : Organisation technique		29
B Annexe B : Exemples de configuration		30

Table des figures

1.1	Logo officiel de Forvis Mazars utilisé pour identifier l'entreprise d'accueil . . .	3
3.1	Cas d'utilisation retenus pour l'application	11
3.2	Cycle de vie fonctionnel d'une détection	11
4.1	Architecture globale de la plateforme	14
4.2	Organisation Docker et séparation du réseau MISP	14
4.3	Workflow LangGraph et boucle de réparation	15
4.4	Workflow d'ingestion MISP	15
4.5	Vue simplifiée du schéma de données	16
5.1	Séquence simplifiée d'une ingestion manuelle	18
5.2	Emplacement prévu pour la capture du tableau de bord	20
5.3	Emplacement prévu pour la capture de la boîte MISP	20
5.4	Emplacement prévu pour la capture des événements CTI	20
5.5	Emplacement prévu pour la capture du tableau de bord IA	21
5.6	Emplacement prévu pour la capture du workflow IA visuel	21
5.7	Emplacement prévu pour la capture de la file des menaces	21
5.8	Emplacement prévu pour la capture de la file de revue	22
5.9	Emplacement prévu pour la capture du catalogue de détections	22
5.10	Emplacement prévu pour la capture de la matrice de couverture ATT&CK	22
5.11	Emplacement prévu pour la capture de la santé système	23
5.12	Workflow de revue humaine et publication locale	23
6.1	Principe de la réparation en session identique	25

Liste des tableaux

3.1	Synthèse des besoins fonctionnels	10
5.1	Principaux domaines API de la solution	17
6.1	Couverture de tests disponible	24

Introduction générale

Les équipes de sécurité reçoivent aujourd’hui un volume important de renseignements sur les menaces. Ces informations peuvent provenir de rapports, d’indicateurs techniques, d’événements MISP ou de sources internes. Leur valeur dépend pourtant de la capacité à les transformer en actions concrètes : comprendre le comportement de l’attaquant, vérifier la couverture existante, produire une logique de détection valide et faire valider le résultat par un analyste.

Dans ce contexte, mon stage d’un mois chez Forvis Mazars Group a porté sur la conception et la réalisation d’un prototype académique intitulé : *Conception et développement d’une plateforme intelligente d’assistance à l’ingénierie de détection basée sur la Cyber Threat Intelligence et l’intelligence artificielle*. L’objectif n’était pas de remplacer l’analyste, mais d’étudier comment l’intelligence artificielle peut assister certaines étapes répétitives de l’ingénierie de détection tout en restant contrôlée par des règles déterministes.

La solution développée repose sur une architecture web complète. Le backend est implémenté avec FastAPI et SQLAlchemy, la persistance avec PostgreSQL, les traitements asynchrones avec Redis et Celery, l’orchestration avec LangGraph et l’interface utilisateur avec Next.js. L’intégration CTI s’appuie sur MISP, tandis que la génération de règles de détection utilise le format Sigma et la validation pySigma. La plateforme comprend également des mécanismes de revue humaine, de détection de doublons, d’analyse de couverture ATT&CK, de vérification de télémétrie et de suivi de santé du système.

Ce rapport présente d’abord le cadre général du stage, puis l’état de l’art nécessaire à la compréhension du sujet. Il détaille ensuite les besoins, la conception, la réalisation technique, les scénarios de validation, les résultats obtenus et les limites du prototype.

Chapitre 1

Cadre du stage et contexte professionnel

1.1 Présentation de Forvis Mazars Group

Forvis Mazars Group est un acteur international du conseil, de l'audit, de la fiscalité et des services aux entreprises. L'organisation s'inscrit dans l'héritage de Mazars, dont l'origine remonte à 1945 à Rouen, et appartient aujourd'hui au réseau mondial Forvis Mazars. Cette évolution traduit une volonté de combiner une présence internationale, une expertise métier reconnue et une approche de proximité adaptée aux besoins des clients [For26a, For26b].

Le groupe intervient auprès d'organisations de tailles et de secteurs variés. Ses activités couvrent notamment l'audit et l'assurance, la fiscalité, le conseil financier, le conseil en organisation, la transformation digitale et les sujets liés aux risques. Cette diversité donne au groupe une position particulière : il accompagne les entreprises non seulement dans leurs obligations réglementaires, mais également dans l'amélioration de leurs processus, la maîtrise de leurs systèmes d'information et la préparation aux risques émergents.

La cybersécurité s'inscrit naturellement dans ce périmètre. Les entreprises dépendent de plus en plus de systèmes numériques interconnectés ; elles doivent donc anticiper les risques, vérifier leurs contrôles, améliorer leur résilience et structurer leurs capacités de détection. Les offres de conseil en cybersécurité de Forvis Mazars portent notamment sur la gouvernance, l'évaluation de maturité, la conformité, la gestion des risques, la préparation aux incidents et l'amélioration des dispositifs de sécurité [For26c]. Le sujet de ce stage est donc cohérent avec les enjeux actuels du groupe : aider les organisations à mieux transformer l'information sur les menaces en actions de sécurité vérifiables.



FIGURE 1.1 – Logo officiel de Forvis Mazars utilisé pour identifier l’entreprise d’accueil

1.2 Contexte du stage

Le stage s’est déroulé sur une durée d’un mois. Cette durée limitée imposait de définir un périmètre réaliste : concevoir une solution démontrable, suffisamment complète pour couvrir une chaîne technique cohérente, mais sans prétendre atteindre le niveau de durcissement attendu d’une plateforme de production. Le travail réalisé doit donc être compris comme un prototype académique orienté ingénierie, permettant de valider des choix d’architecture et d’observer les difficultés concrètes d’une automatisation de détection.

Le thème choisi relie trois domaines : la Cyber Threat Intelligence, l’ingénierie de détection et l’intelligence artificielle. La CTI fournit des événements et des indicateurs relatifs aux menaces. L’ingénierie de détection transforme ces informations en règles exploitables par un SOC. L’intelligence artificielle peut assister certaines tâches, comme l’extraction de comportements ou la rédaction initiale d’une règle, mais elle introduit également des risques : hallucinations, manque de justification, incohérences techniques ou confiance excessive.

1.3 Sujet et problématique

Le sujet du stage est la conception et le développement d’une plateforme intelligente d’assistance à l’ingénierie de détection basée sur la CTI et l’intelligence artificielle. La plateforme reçoit des événements MISP, les transforme en événements internes, extrait des comportements suspects, les relie au référentiel MITRE ATT&CK, vérifie la couverture existante, analyse la disponibilité de la télémétrie, génère des propositions Sigma et soumet ces propositions à une revue humaine.

La problématique peut être formulée ainsi :

Comment assister l’ingénieur de détection dans la transformation de renseignements CTI en règles Sigma exploitables, tout en conservant une validation déterministe, une traçabilité complète et une décision finale humaine ?

Cette formulation met l'accent sur un point essentiel : l'objectif n'est pas de donner à l'IA un rôle autonome de décision, mais de l'intégrer dans un processus contrôlé. Dans un contexte SOC, une règle de détection erronée peut produire du bruit, manquer une attaque réelle ou donner une fausse impression de couverture. La solution devait donc être conçue autour de garde-fous techniques.

1.4 Objectifs poursuivis

Les objectifs du stage se répartissent en trois niveaux. Le premier niveau concerne l'ingestion et la structuration de la CTI. Il s'agit d'utiliser MISP comme source réelle, de récupérer les événements via son API et de les normaliser dans une base relationnelle. Le deuxième niveau concerne l'orchestration de l'analyse : extraction de comportements, mapping ATT&CK, vérification de couverture, contrôle de télémétrie et décision de politique. Le troisième niveau concerne la production contrôlée d'une règle : génération Sigma, validation pySigma, détection de doublons, réparation éventuelle, scoring et revue analyste.

Ces objectifs ont été choisis pour représenter une chaîne d'ingénierie de détection réaliste. Une règle ne doit pas être produite uniquement parce qu'un indicateur existe. Elle doit correspondre à un comportement observable, être rattachée à une technique compréhensible, reposer sur une télémétrie disponible et ne pas dupliquer une détection déjà présente.

1.5 Organisation du rapport

Le chapitre suivant présente les fondements techniques nécessaires à la compréhension du projet. Le troisième chapitre formalise les besoins fonctionnels et non fonctionnels. Les chapitres quatre et cinq décrivent respectivement la conception et la réalisation. Le sixième chapitre présente la validation, les résultats obtenus et les limites. La conclusion revient enfin sur les apprentissages et les perspectives.

Chapitre 2

État de l’art et fondements scientifiques

2.1 Cyber Threat Intelligence et exploitation opérationnelle

La Cyber Threat Intelligence regroupe les informations permettant de comprendre les acteurs malveillants, leurs infrastructures, leurs outils, leurs techniques et leurs objectifs. Dans un contexte opérationnel, elle ne se limite pas à la collecte d’indicateurs. Sa valeur dépend de la capacité à la relier à des décisions concrètes : prioriser une investigation, enrichir une alerte, orienter une chasse aux menaces ou produire une détection.

MISP occupe une place importante dans cet écosystème. La plateforme permet de structurer des événements, des attributs, des objets et des tags afin de faciliter le partage de renseignements [MIS26]. Le choix de MISP dans ce travail se justifie par son usage courant dans les communautés de cybersécurité et par la disponibilité d’une API permettant de construire un flux d’ingestion réaliste. Cette approche évite de travailler sur des données artificiellement insérées en base et oblige à traiter les problèmes habituels d’une intégration : authentification, format des événements, redondance et disponibilité du service.

2.2 SOC, SIEM et ingénierie de détection

Un Security Operations Center a pour objectif de surveiller les systèmes d’information, détecter les comportements suspects, qualifier les alertes et coordonner la réponse. Les SIEM centralisent les journaux et exécutent des règles de corrélation ou de détection.

Cependant, la qualité d'un SOC ne dépend pas uniquement de l'outil SIEM. Elle dépend fortement de l'ingénierie de détection : compréhension de la menace, choix des sources de logs, écriture de règles pertinentes, réduction du bruit, validation et maintenance.

L'ingénierie de détection est donc une discipline de traduction. Elle transforme une connaissance sur l'adversaire en logique observable. Cette traduction n'est pas automatique. Un indicateur réseau peut être bloqué directement, mais un comportement comme l'exécution de PowerShell encodé ou l'accès à LSASS nécessite de comprendre les traces disponibles, les conditions de déclenchement et les risques de faux positifs. C'est précisément cette difficulté que le projet cherche à adresser.

2.3 Threat Hunting et approche comportementale

Le Threat Hunting complète la détection classique en formulant des hypothèses sur les comportements adverses et en les recherchant activement dans la télémétrie. Cette démarche favorise une approche comportementale plutôt qu'une dépendance excessive aux indicateurs statiques. Dans le projet, cette idée apparaît dans la séparation entre les observables extraits de MISP et les comportements exploités par le workflow.

L'utilisation du référentiel MITRE ATT&CK renforce cette orientation. ATT&CK fournit une base de connaissances décrivant tactiques et techniques adverses [MIT26a]. Il permet de relier une proposition de détection à une technique connue, d'organiser la couverture et de justifier les décisions prises pendant l'analyse.

2.4 Sigma et validation indépendante du SIEM

Sigma propose un format générique de règles de détection indépendant d'un SIEM particulier [Sig26b]. Ce choix est pertinent pour un projet académique, car il permet de produire un contenu lisible, structuré et portable. Néanmoins, une règle Sigma générée par l'IA ne peut pas être considérée correcte sans validation. Le format doit être conforme, les champs doivent être cohérents et la règle doit pouvoir être compilée vers un backend cible.

pySigma joue ce rôle de contrôle [Sig26a]. Dans la solution réalisée, il permet de vérifier la syntaxe Sigma et de compiler la règle vers une sortie Splunk. Ce mécanisme introduit une validation déterministe dans un processus où l'IA peut proposer un contenu imparfait.

2.5 IA humaine dans la boucle

L'intelligence artificielle peut accélérer certaines tâches de cybersécurité, mais elle soulève des questions de gouvernance. Dans un processus de détection, une réponse plausible mais non supportée par des éléments factuels peut être dangereuse. L'approche human-in-the-loop consiste à maintenir l'humain dans la décision, en particulier lorsque le résultat a un impact opérationnel.

Le projet applique cette idée en imposant une revue analyste avant publication. L'IA peut extraire, proposer et réparer, mais elle ne peut pas approuver seule une détection. Cette séparation est cohérente avec les principes d'IA explicable et gouvernée : le système doit produire des traces, expliquer les facteurs de score et permettre à un analyste de comprendre pourquoi une proposition est recommandée, bloquée ou rejetée.

2.6 IA explicable, confiance et gouvernance

La littérature sur l'IA explicable insiste sur la nécessité de rendre les décisions compréhensibles par les utilisateurs humains [ADRDS⁺20]. Dans ce projet, cette exigence se traduit par des watchers, une mémoire structurée, des scores séparés et des résultats persistés. La confiance associée au candidat n'est pas suffisante ; elle doit être complétée par un score de trust déterministe fondé sur les validations techniques.

Cette séparation répond à un risque classique : une forte confiance déclarée par l'IA pourrait masquer une règle invalide ou non supportée par l'évidence. En distinguant confiance et trust, la solution empêche une sortie IA persuasive de contourner la validation Sigma, l'analyse de télémétrie, la détection de doublons ou les règles de sécurité.

2.7 Cadres de référence complémentaires

Le NIST Cybersecurity Framework organise les activités de cybersécurité autour de fonctions telles que Identify, Protect, Detect, Respond et Recover [Nat24]. Le projet se situe principalement dans la fonction Detect, avec des liens vers Identify à travers la connaissance des actifs de télémétrie et vers Respond à travers la préparation de contenus exploitables par les analystes.

MITRE D3FEND apporte une perspective défensive en décrivant des contre-mesures et techniques de défense [MIT26b]. Même si le projet n'implémente pas directement D3FEND, ce référentiel confirme l'intérêt de relier la connaissance offensive à des mécanismes défensifs structurés. Cette logique guide la conception de la matrice de couverture

et du cycle de vie des détections.

Chapitre 3

Analyse des besoins et cahier des charges

3.1 Expression du besoin

Le besoin initial peut être résumé comme la construction d'une chaîne permettant de passer d'un événement CTI à une proposition de détection contrôlée. Ce besoin répond à une réalité opérationnelle : les analystes disposent souvent d'informations sur les menaces, mais la transformation de ces informations en règles utiles demande du temps, une connaissance des techniques adverses et une bonne compréhension de la télémétrie disponible.

La solution devait donc assister l'analyste sans supprimer son rôle. Elle devait accélérer les étapes répétitives, rendre visibles les décisions prises par le workflow et empêcher qu'une règle non validée soit considérée comme prête à l'emploi. Cette orientation a guidé l'ensemble du cahier des charges.

3.2 Besoins fonctionnels

Les besoins fonctionnels couvrent l'ensemble du cycle de vie de la détection. L'application devait permettre à un utilisateur authentifié de consulter les événements MISP, de lancer une ingestion manuelle, de déclencher un traitement en masse ou de configurer une ingestion planifiée. Après normalisation, chaque événement devait être associé à un workflow de détection.

Le workflow devait extraire un comportement, proposer un mapping ATT&CK, vérifier la couverture existante, analyser la télémétrie disponible et décider s'il fallait générer une nouvelle règle. Lorsqu'une règle était nécessaire, l'application devait produire une

proposition Sigma, la valider, calculer des scores, détecter les doublons et, si possible, réparer les erreurs. La dernière étape devait rester une revue analyste, avec possibilité d'approuver, de demander des changements ou de rejeter.

TABLE 3.1 – Synthèse des besoins fonctionnels

Domaine	Besoin couvert
Accès	Authentification locale, jetons JWT, rôles administrateur et analyste.
CTI	Consultation, normalisation et ingestion d'événements MISP.
Workflow	Exécution asynchrone et traçable d'un graphe de détection.
IA	Extraction de comportements, génération de règles, réparation bornée.
Contrôles	Watchers, validation Sigma, pySigma, doublons, télémétrie et politique.
Revue	File de propositions, historique des révisions et décision humaine.
Suivi	Santé système, consommation MISP, sessions IA et timeline des graphes.

3.3 Besoins non fonctionnels

La traçabilité constitue le besoin non fonctionnel le plus important. Une proposition de détection ne devait pas être un simple résultat final. Il fallait pouvoir retrouver l'événement source, le comportement extrait, les mappings ATT&CK, les validations, les erreurs, les réparations, les scores et la décision finale. Cette exigence explique la richesse du modèle de données et la persistance des noeuds de graphe.

La reproductibilité était également nécessaire. Le projet étant destiné à une démonstration académique, l'environnement devait pouvoir être relancé localement. Docker Compose a donc été retenu pour assembler les services principaux. Les traitements longs ont été confiés à Celery afin de ne pas bloquer l'interface et de mieux représenter un comportement de plateforme SOC.

3.4 Contraintes de sécurité

La cybersécurité impose de ne pas traiter les sorties IA comme des vérités. Le cahier des charges a donc imposé plusieurs garde-fous : validation Pydantic des entrées API, authentification des routes sensibles, configuration des secrets par variables d'environnement, vérification des règles Sigma, détection de doublons et approbation humaine avant publication. Ces contraintes ne transforment pas l'application en solution prête pour une production d'entreprise, mais elles donnent une base saine pour un démonstrateur.

3.5 Cas d'utilisation

La figure 3.1 présente les principaux acteurs et interactions. L'analyste SOC utilise surtout la file des menaces, la revue et le catalogue. L'administrateur intervient davantage sur la configuration, la santé système et l'ingestion planifiée.

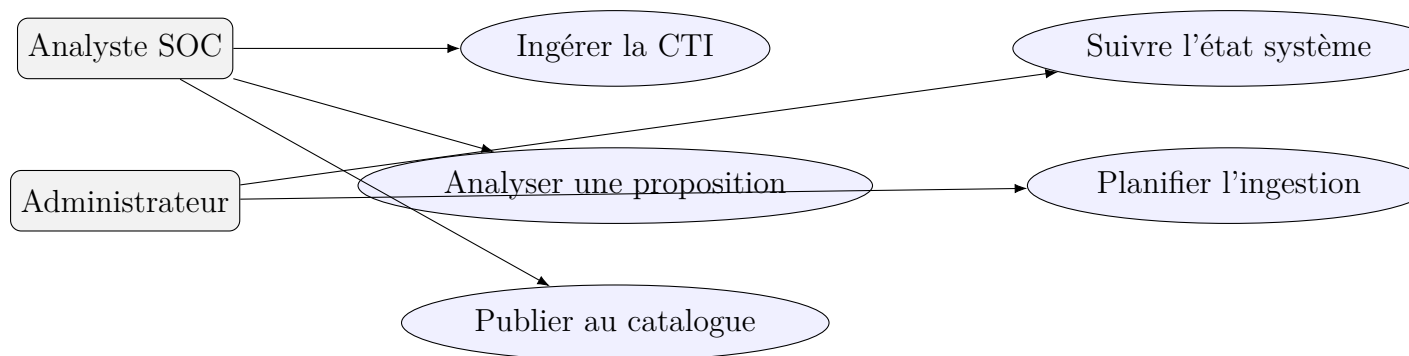


FIGURE 3.1 – Cas d'utilisation retenus pour l'application

3.6 Cycle de vie attendu d'une détection

Une règle de détection suit un cycle de vie progressif. Elle naît d'un renseignement, devient un comportement, puis une proposition technique. Elle est ensuite validée, éventuellement corrigée, et enfin soumise à l'analyste. Ce cycle évite de confondre une hypothèse de détection avec une règle prête à être publiée.

CTI → Comportement → CandidatSigma → Validation → Revue → Catalogue

FIGURE 3.2 – Cycle de vie fonctionnel d'une détection

3.7 Critères d'acceptation

Le système devait être jugé sur sa capacité à démontrer un flux complet et traçable. Les critères principaux étaient l'ingestion MISP réelle, la création d'un workflow, la persistance des étapes, l'application effective des watchers, la validation pySigma, la détection de doublons, la possibilité de réparation et le maintien de l'analyste comme autorité finale. Les limites connues devaient également être documentées sans exagération.

Chapitre 4

Conception et architecture de la solution

4.1 Principes de conception

La conception s'appuie sur trois principes. Le premier est la séparation des responsabilités : l'interface affiche et déclenche, le backend orchestre, les services déterministes vérifient et la base conserve les preuves. Le deuxième est la traçabilité : chaque événement important doit laisser une trace exploitable. Le troisième est la prudence vis-à-vis de l'IA : aucune sortie générée ne doit contourner les validations techniques ou la revue humaine.

Ces principes expliquent les choix d'architecture. Une application monolithique simple aurait été plus rapide à écrire, mais elle aurait mal représenté les contraintes d'un système SOC. À l'inverse, une architecture trop distribuée aurait dépassé le périmètre d'un stage d'un mois. La solution retenue cherche donc un équilibre : plusieurs services clairement séparés, mais orchestrés par Docker Compose dans un environnement local.

4.2 Architecture générale

L'application repose sur une interface Next.js, un backend FastAPI, une base PostgreSQL, Redis, Celery et un proxy Nginx. MISP est intégré comme source CTI externe accessible par API. La figure 4.1 montre les échanges principaux.

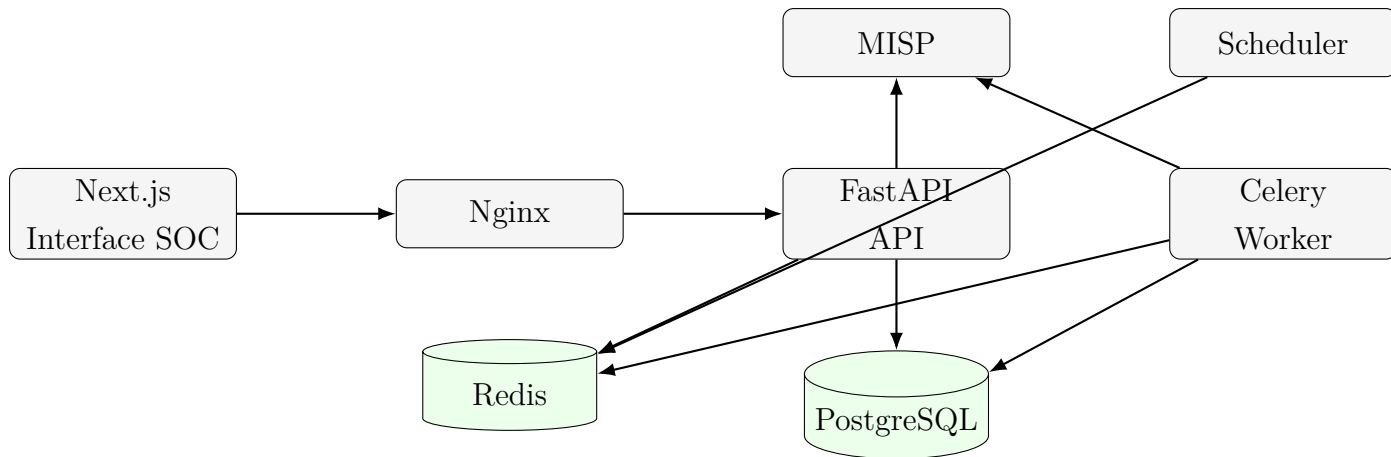


FIGURE 4.1 – Architecture globale de la plateforme

FastAPI a été choisi pour sa rapidité de développement, sa validation Pydantic et sa documentation automatique. PostgreSQL a été retenu car le modèle mélange des relations fortes et des données JSONB, ce qui convient aux événements CTI et aux traces IA. Redis et Celery répondent au besoin d'exécution asynchrone : une analyse CTI peut nécessiter plusieurs étapes et ne doit pas bloquer une requête HTTP. Next.js offre une base adaptée à une interface SOC réactive et typée.

4.3 Architecture Docker

Docker Compose permet de déployer de manière reproductible les services nécessaires au démonstrateur. Les services applicatifs principaux sont séparés de l'environnement MISP, qui fonctionne sous profil dédié. Cette séparation répond à deux besoins : limiter la complexité au démarrage lorsque MISP n'est pas nécessaire, et isoler les services MISP sur un réseau spécifique.

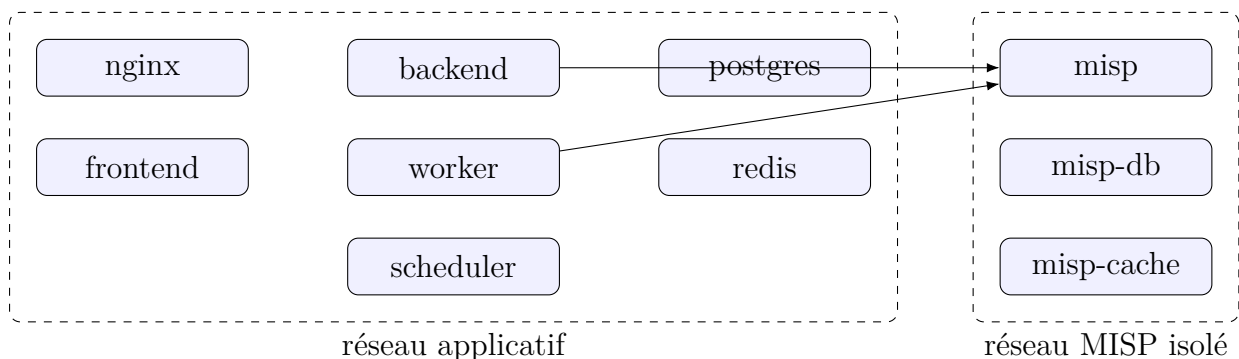


FIGURE 4.2 – Organisation Docker et séparation du réseau MISP

4.4 Conception du workflow IA

L'usage de LangGraph est motivé par le besoin de représenter explicitement le chemin suivi par un événement. Contrairement à une suite de fonctions appelée linéairement, un graphe permet de modéliser des routes terminales différentes : déjà couvert, gap de visibilité, rejet, réparation ou revue. Il permet également de reprendre un traitement et de conserver une trace de chaque noeud.

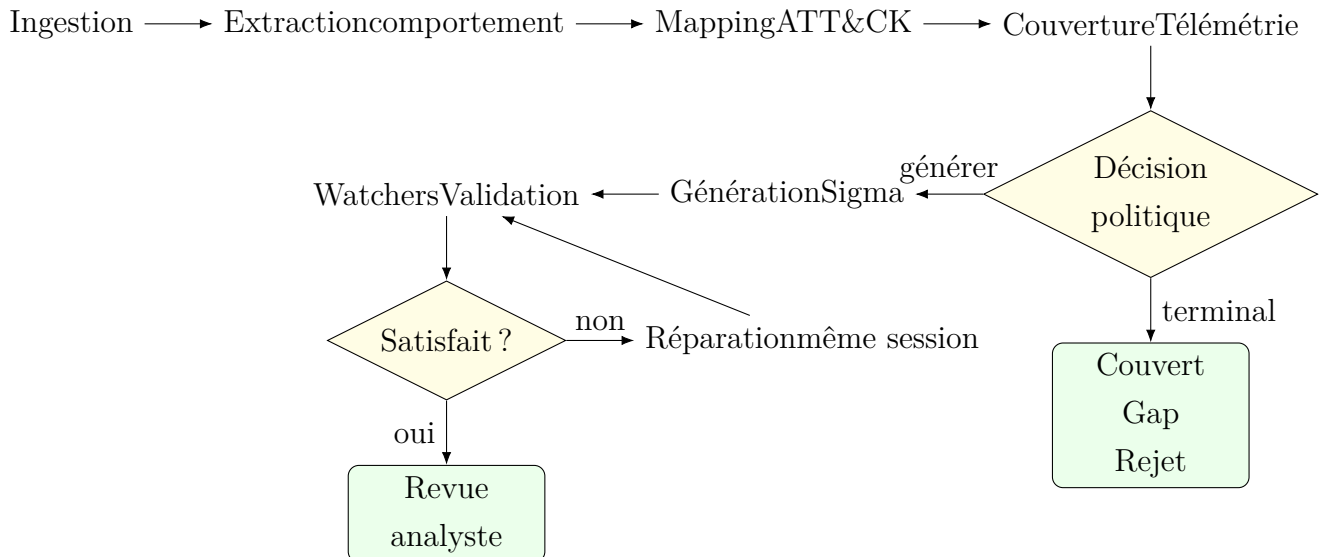


FIGURE 4.3 – Workflow LangGraph et boucle de réparation

4.5 Ingestion MISP

L'ingestion MISP suit une séquence contrôlée. L'utilisateur ou le scheduler demande l'évaluation des événements, le backend interroge MISP, vérifie l'idempotence, crée ou ignore l'événement CTI, puis enregistre un statut de consommation. Cette granularité est importante car une ingestion en lot doit conserver un résultat individuel pour chaque événement évalué.

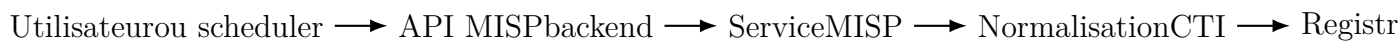


FIGURE 4.4 – Workflow d'ingestion MISP

4.6 Modèle de données

Le modèle de données a été conçu pour relier les sources, les décisions et les résultats. Les événements CTI alimentent les workflows. Les workflows produisent des comportements,

des mappings, des analyses de couverture, des résultats de visibilité, des propositions et des révisions. Les tables de sessions IA, watchers et événements de confiance permettent de reconstruire la logique de raisonnement sans exposer de chaîne de pensée.

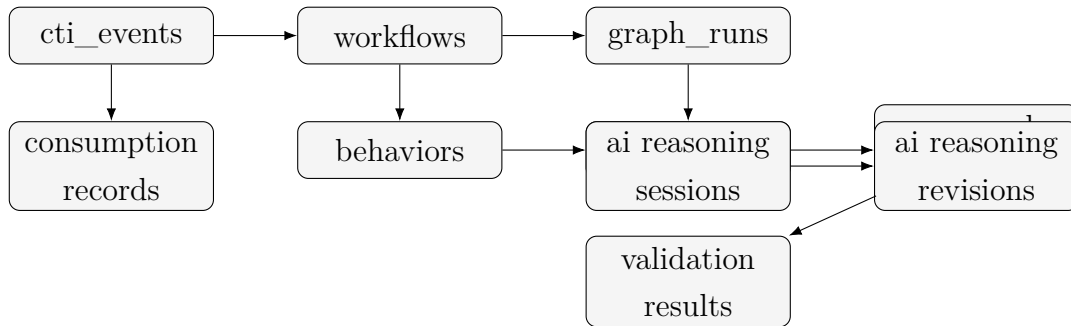


FIGURE 4.5 – Vue simplifiée du schéma de données

4.7 Rôle des watchers et des scores

Les watchers ont été introduits pour matérialiser des contrôles lisibles : conformité du schéma, présence d'évidence, cohérence ATT&CK, validité Sigma, niveau de confiance, doublons, télémétrie, hallucination et sécurité. Ils répondent à une exigence de gouvernance de l'IA. Un candidat n'est pas accepté parce qu'il semble correct ; il est accepté seulement si les contrôles pertinents sont satisfaits ou si les risques restants justifient une revue.

La séparation entre confiance et trust complète cette logique. La confiance peut représenter la qualité estimée d'une proposition, tandis que le trust agrège des facteurs déterministes. Ainsi, une validation Sigma échouée, un doublon exact ou une absence de télémétrie ne peuvent pas être compensés par une note élevée produite par l'IA.

Chapitre 5

Réalisation de la plateforme

5.1 Réalisation du backend

Une architecture backend a été développée avec FastAPI. Ce choix a permis de combiner rapidité de développement, validation forte des données via Pydantic et séparation claire entre routes, schémas, services et modèles. Les endpoints sont regroupés par domaine fonctionnel : authentification, CTI, MISP, graphes, IA, propositions, revue, télémétrie, catalogue et opérations.

L'application met en oeuvre une authentification par jetons JWT. Les utilisateurs disposent de rôles locaux, principalement administrateur et analyste. Cette solution reste adaptée au contexte de démonstration, mais elle ne remplace pas une intégration SSO ou un RBAC d'entreprise. Elle permet cependant de matérialiser une frontière importante : les opérations sensibles ne sont pas publiques.

TABLE 5.1 – Principaux domaines API de la solution

Domaine	Rôle dans l'application
Authentification	Connexion, renouvellement des jetons et identification de l'utilisateur courant.
CTI	Consultation des événements normalisés et lancement de workflows.
MISP	Listing, détail, ingestion unitaire, ingestion en lot et planification.
Graphe	Consultation des runs et des timelines de noeuds.
IA	Tableau de bord, sessions, workflow visuel et consommation.

Revue	Consultation des propositions, révisions, validations et actions analyste.
Catalogue	Consultation et import de détections approuvées.
Opérations	Santé système, paramètres et artefacts de déploiement.

5.2 Persistance et migrations

PostgreSQL a été choisi parce que le domaine nécessite à la fois des relations explicites et des données semi-structurées. Les événements MISP, les réponses IA et les résultats de watchers contiennent naturellement des structures JSON. Le type JSONB permet de conserver ces détails tout en maintenant des liens relationnels entre événements, workflows, propositions et validations.

Les migrations Alembic structurent l'évolution du schéma. La première migration crée le socle applicatif. Les migrations suivantes ajoutent les métadonnées de politique, l'audit ATT&CK, les contraintes d'idempotence, le moteur de raisonnement IA et les champs de conformité demandés pour le suivi des révisions et de la consommation. Cette progression reflète l'évolution du besoin : partir d'une chaîne CTI-to-detection, puis renforcer progressivement la traçabilité.

5.3 Ingestion MISP et consommation

L'intégration MISP utilise les variables `CTI_MISP_URL`, `CTI_MISP_API_KEY` et `CTI_MISP_VERIFY_TLS`. La plateforme distingue plusieurs états : non configuré, inaccessible, échec d'authentification et sain. Cette distinction est importante car un simple état « configuré » ne suffit pas à prouver que l'ingestion fonctionne réellement.

L'ingestion peut être déclenchée manuellement pour un événement, exécutée en lot pour tous les nouveaux événements ou planifiée. Chaque événement évalué produit un enregistrement de consommation. Les statuts consommé, dupliqué, ignoré et échoué permettent de diagnostiquer les erreurs sans modifier directement la base de données.



FIGURE 5.1 – Séquence simplifiée d'une ingestion manuelle

5.4 Orchestration asynchrone

Les traitements d'ingénierie de détection sont exécutés par Celery. Cette décision évite que l'utilisateur attende la fin complète d'une analyse dans une requête HTTP. Elle permet aussi de représenter plus fidèlement un fonctionnement SOC, où l'arrivée de nouveaux renseignements déclenche des jobs en arrière-plan. Redis joue le rôle de broker et de support d'orchestration.

Le scheduler gère le polling MISP selon la configuration choisie : désactivé, intervalle, horaire, quotidien, hebdomadaire ou exécution unique. L'interface permet de modifier cette configuration et le backend calcule les échéances de polling.

5.5 Réalisation du moteur de raisonnement

Le moteur de raisonnement combine des services déterministes et des appels IA. Le mode fixture rend les démonstrations reproductibles, tandis qu'une intégration DeepSeek est prévue pour un fournisseur réel lorsque les credentials sont configurés. Les interactions IA sont persistées avec le provider, le modèle, la version de prompt, les entrées, les sorties structurées, les latences et les erreurs éventuelles.

La boucle de réparation fonctionne dans une même session de raisonnement. Lorsqu'un candidat échoue sur un watcher ou une validation réparable, une nouvelle révision est créée avec un lien parent. La mémoire structurée transmet les erreurs précédentes, les mappings acceptés et les stratégies à éviter. Cette approche répond à un besoin concret : ne pas produire plusieurs réponses indépendantes qui répètent les mêmes erreurs.

5.6 Génération et validation Sigma

La génération Sigma transforme le comportement extrait en règle structurée. La validation ne se limite pas au format YAML. Elle vérifie la conformité Sigma, la compilation pySigma vers Splunk, la qualité, le mapping ATT&CK, la télémétrie et le risque de doublon. Une règle invalide ne peut donc pas avancer comme si elle était prête pour la revue.

Le pipeline de validation constitue l'un des points clés de la solution. Il transforme une sortie IA en objet vérifiable. En cas d'échec réparable, il déclenche la boucle de correction. En cas d'échec bloquant, il route vers une issue terminale adaptée.

5.7 Interface SOC

L'interface a été conçue comme un espace de travail pour analyste. Elle ne se présente pas comme une page marketing, mais comme un ensemble de vues opérationnelles : tableau de bord, file des menaces, sessions IA, workflow IA, runs, propositions, revue, catalogue, matrice ATT&CK, télémétrie, santé système et paramètres.

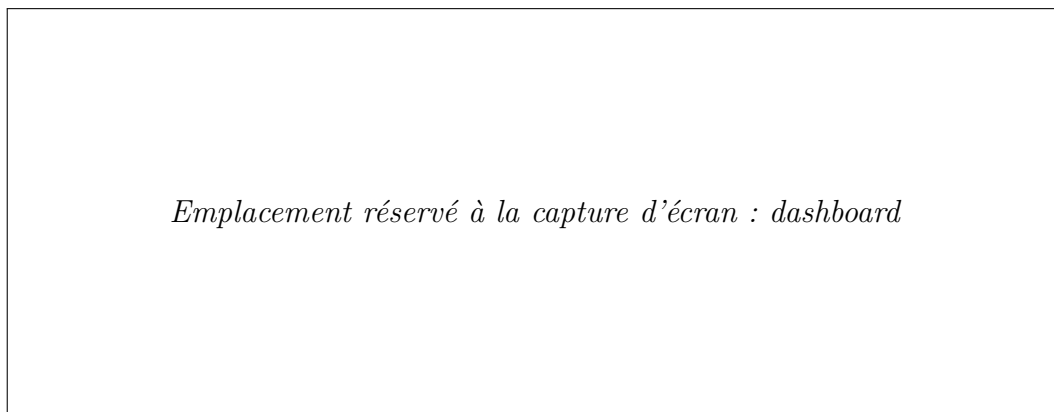


FIGURE 5.2 – Emplacement prévu pour la capture du tableau de bord

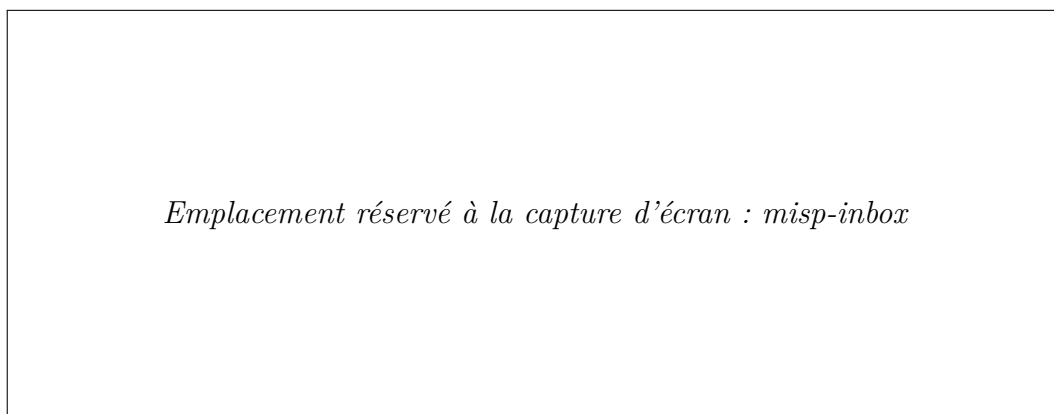


FIGURE 5.3 – Emplacement prévu pour la capture de la boîte MISP

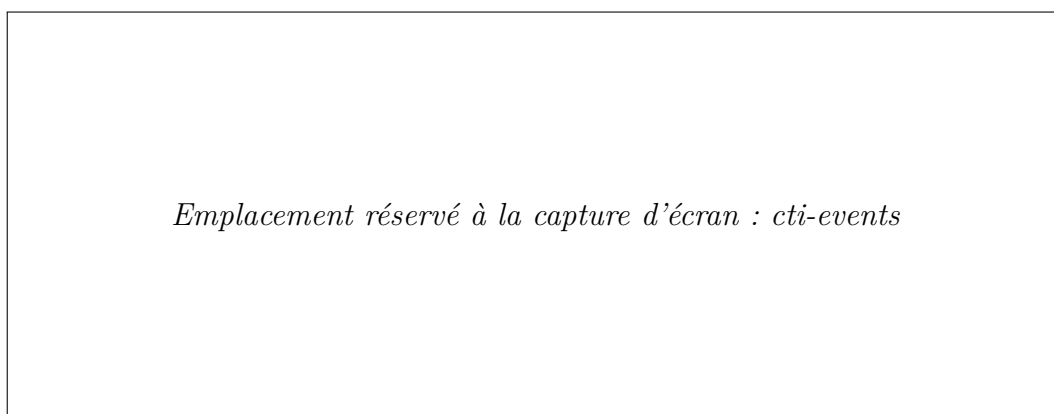
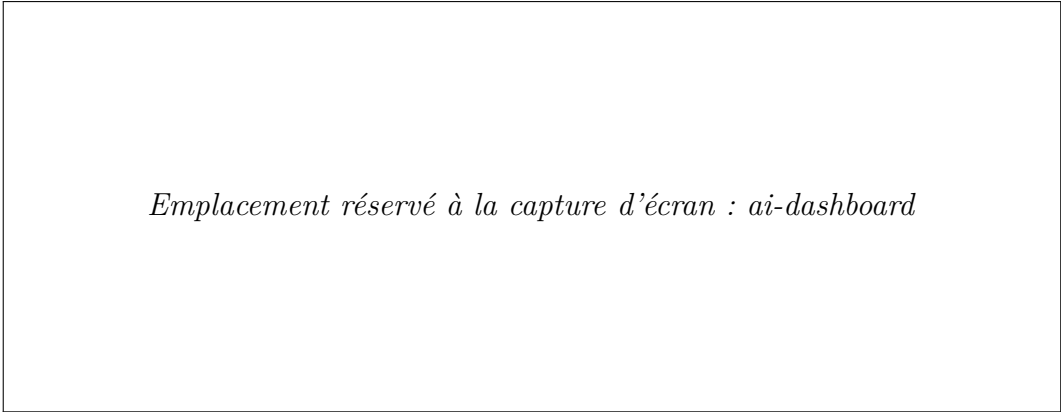
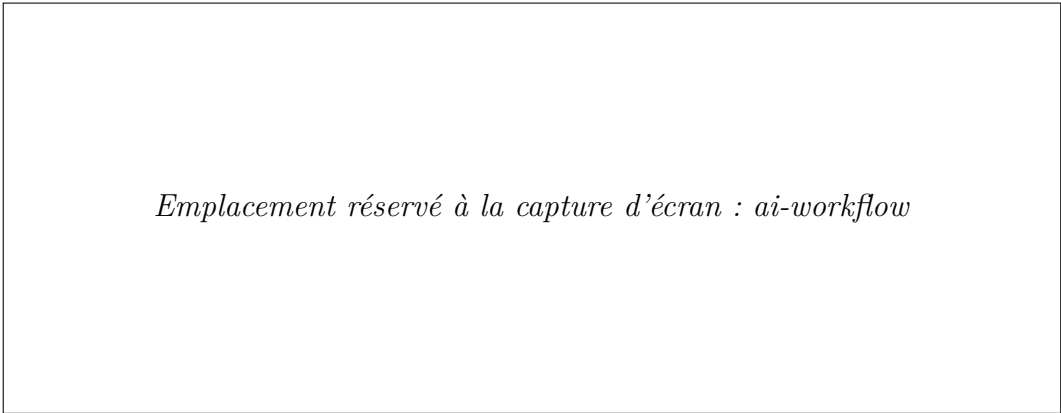


FIGURE 5.4 – Emplacement prévu pour la capture des événements CTI



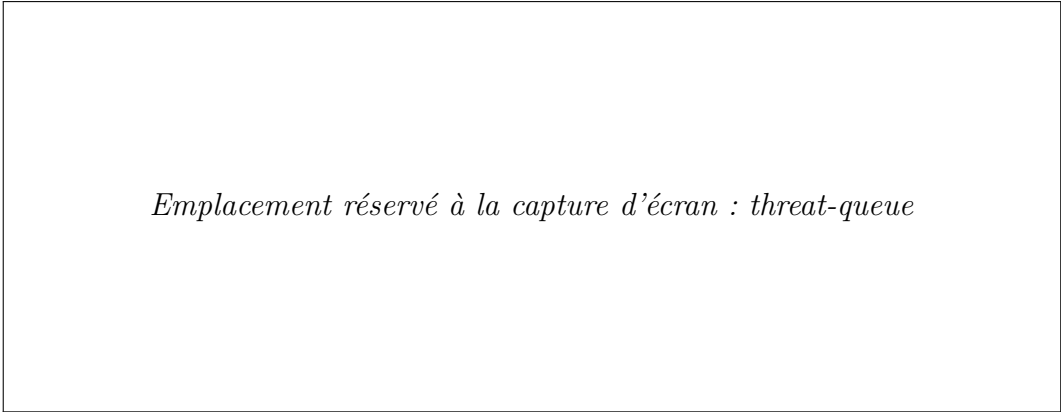
Emplacement réservé à la capture d'écran : ai-dashboard

FIGURE 5.5 – Emplacement prévu pour la capture du tableau de bord IA



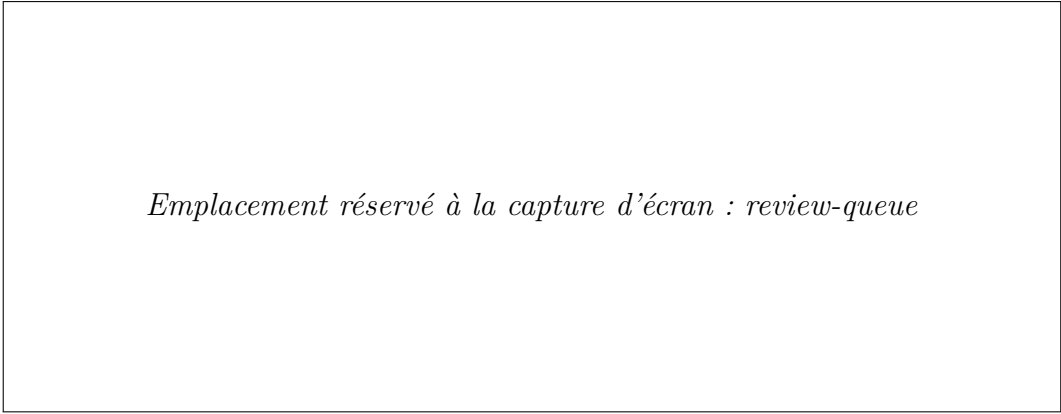
Emplacement réservé à la capture d'écran : ai-workflow

FIGURE 5.6 – Emplacement prévu pour la capture du workflow IA visuel



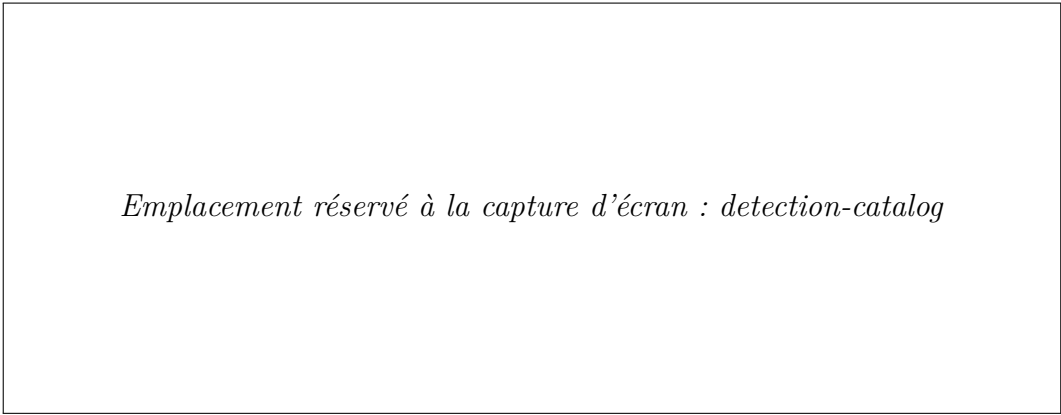
Emplacement réservé à la capture d'écran : threat-queue

FIGURE 5.7 – Emplacement prévu pour la capture de la file des menaces



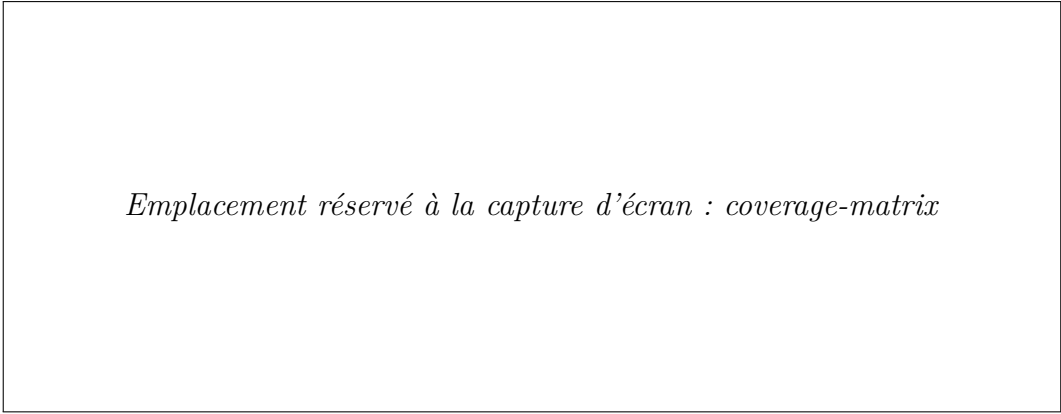
Emplacement réservé à la capture d'écran : review-queue

FIGURE 5.8 – Emplacement prévu pour la capture de la file de revue



Emplacement réservé à la capture d'écran : detection-catalog

FIGURE 5.9 – Emplacement prévu pour la capture du catalogue de détections



Emplacement réservé à la capture d'écran : coverage-matrix

FIGURE 5.10 – Emplacement prévu pour la capture de la matrice de couverture ATT&CK

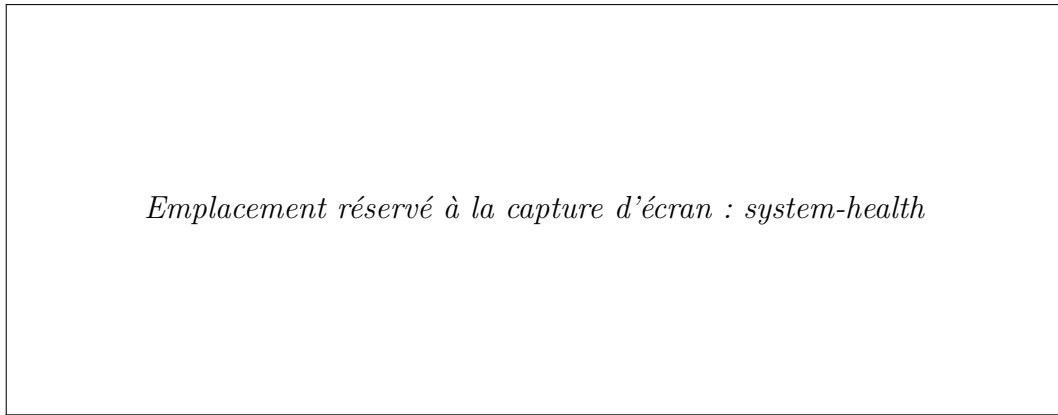


FIGURE 5.11 – Emplacement prévu pour la capture de la santé système

5.8 Cycle de revue et publication

La revue humaine est obligatoire. Cette décision est volontaire : dans un système de détection, approuver une règle revient à accepter un impact opérationnel. L'analyste peut approuver, demander des changements ou rejeter. Lorsqu'une proposition est approuvée, la solution génère un artefact local et publie la détection dans le catalogue interne. Le déploiement vers un SIEM réel reste hors périmètre.

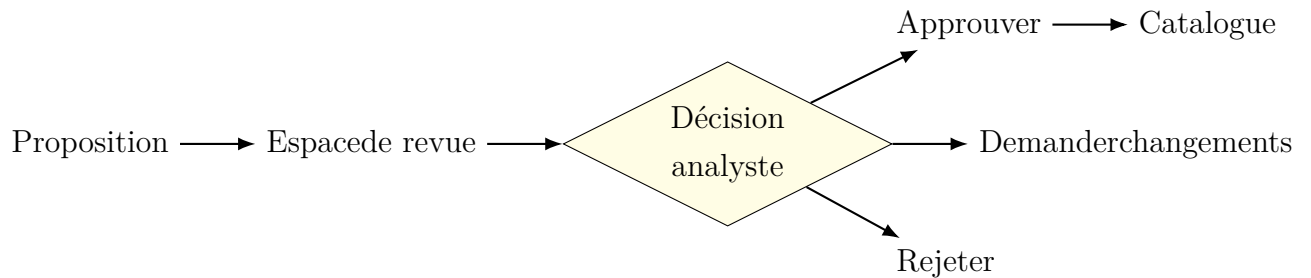


FIGURE 5.12 – Workflow de revue humaine et publication locale

Chapitre 6

Validation, résultats et limites

6.1 Méthode de validation

La validation a été organisée autour des scénarios attendus par la plateforme : ingestion MISP, exécution de workflow, génération de proposition, validation Sigma, réparation, détection de doublons, gap de visibilité et revue humaine. Cette méthode est plus pertinente qu’une simple vérification d’interface, car elle confirme que les états sont persistés et que les décisions du workflow ont un effet réel.

Les tests backend couvrent les services déterministes, les contrats IA, le fournisseur DeepSeek, les tâches Celery, la connectivité MISP et le moteur de raisonnement. Ils ne couvrent pas encore l’ensemble de l’interface frontend, ce qui constitue une limite importante.

TABLE 6.1 – Couverture de tests disponible

Fichier	Objectif
<code>test_deterministic_services.py</code>	Vérifier couverture, télémétrie, politique, Sigma, doublons et publication locale.
<code>test_ai_contracts.py</code>	Stabiliser les schémas, prompts et contrats de l’abstraction IA.
<code>test_deepseek_provider.py</code>	Contrôler l’intégration du fournisseur DeepSeek lorsque configuré.
<code>test_celery_contracts.py</code>	Vérifier les contrats des traitements asynchrones.
<code>test_misp_connectivity.py</code>	Tester la configuration et les états de connectivité MISP.
<code>test_reasoning_engine.py</code>	Couvrir les révisions, watchers, scores, recommandations et consommation.
<code>test_real_ai_automation_contracts.py</code>	Valider des scénarios coeur de l’automatisation IA.

6.2 Scénarios fonctionnels vérifiés

Les scénarios vérifiés montrent que la chaîne principale est cohérente. Une intelligence MISP peut être consommée via l'API, transformée en événement CTI, associée à un workflow, puis traitée par le graphe. Le système conserve les runs, les noeuds, les sessions IA, les révisions et les résultats de validation.

La réparation dans une même session a également été prévue et testée dans le moteur de raisonnement. Cette propriété est importante : une révision corrective ne doit pas être considérée comme une nouvelle tentative indépendante, mais comme la suite d'un raisonnement qui a mémorisé les erreurs précédentes.

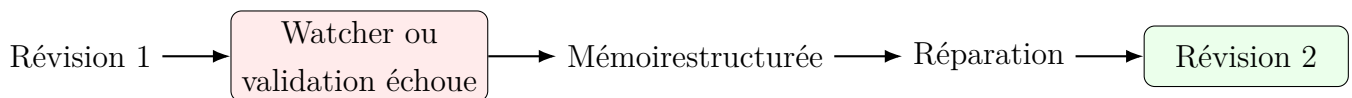


FIGURE 6.1 – Principe de la réparation en session identique

6.3 Résultats obtenus

La réalisation obtenue démontre une chaîne CTI-to-detection complète à l'échelle locale. L'application sait récupérer des événements MISP, éviter des doublons, créer des workflows, piloter une analyse, générer des propositions, appliquer des watchers, valider Sigma, calculer des scores, conserver les révisions et présenter les résultats dans une interface analyste.

Le résultat le plus significatif n'est pas uniquement la génération d'une règle Sigma. Il réside dans l'encadrement de cette génération. L'IA intervient dans un environnement contraint où chaque sortie est vérifiée, historisée et soumise à un analyste. Ce choix rend la solution plus crédible qu'une génération directe de détection par prompt unique.

6.4 Difficultés rencontrées

L'intégration MISP a représenté une difficulté importante. MISP fonctionne avec sa propre configuration, son interface web, ses dépendances et ses clés API. Un problème d'authentification peut faire échouer l'ingestion même si le service web est accessible. Il a donc fallu distinguer la configuration, la connectivité, l'authentification et le polling.

La gestion des doublons a également nécessité une attention particulière. Les tests d'ingestion peuvent produire des événements redondants, et une plateforme d'ingénierie de détection ne doit pas créer plusieurs propositions équivalentes pour le même comporte-

ment. Des mécanismes d'idempotence, de fingerprinting et de statuts de consommation ont été introduits pour maîtriser ce problème.

La troisième difficulté concerne l'IA. Il est relativement simple de demander à un modèle de produire une règle ; il est beaucoup plus difficile de prouver pourquoi cette règle est acceptable ou pourquoi elle doit être corrigée. Les watchers, la mémoire de session et les scores séparés ont été conçus pour répondre à cette difficulté.

6.5 Limites identifiées

La solution n'est pas prête pour un déploiement de production. L'authentification reste locale et ne couvre pas les exigences d'une entreprise mature. L'observabilité ne comprend pas de stack complète de métriques, d'alerting ou de dashboards techniques. Les tests frontend automatisés sont absents. Le mode IA réel dépend d'une clé externe et n'a pas vocation à être garanti dans tous les environnements de démonstration. Enfin, la publication se limite à des artefacts locaux et au catalogue interne ; aucun push vers un SIEM réel n'est implémenté.

Ces limites n'annulent pas la valeur du travail réalisé. Elles clarifient le périmètre : la plateforme démontre une approche d'ingénierie contrôlée et traçable, mais elle nécessite un durcissement avant tout usage opérationnel sensible.

6.6 Bilan

Le bilan technique est positif pour un stage d'un mois. Le projet combine plusieurs composantes qui sont rarement réunies dans un simple exercice académique : CTI, MISP, ATT&CK, Sigma, pySigma, orchestration asynchrone, interface SOC, validation déterministe et boucle IA gouvernée. La principale contribution consiste à montrer comment l'IA peut être intégrée comme assistant, tout en maintenant des contrôles explicites et une autorité humaine finale.

Conclusion générale

Ce stage d'un mois chez Forvis Mazars Group m'a permis de réaliser un prototype complet autour de la transformation de renseignements CTI en propositions de détection. Ce travail m'a amené à combiner des compétences de développement backend, frontend, base de données, orchestration asynchrone, cybersécurité et documentation technique.

La plateforme obtenue intègre MISP, normalise des événements CTI, exécute un workflow LangGraph, utilise une abstraction IA, génère des candidats Sigma, applique des watchers, valide avec pySigma, détecte les doublons et soumet les propositions à une revue humaine. Les pages frontend permettent de visualiser la file des menaces, les sessions IA, les runs de graphe, les propositions, la matrice ATT&CK, la télémétrie, la santé système et le catalogue.

Le principal apprentissage du stage est que l'intelligence artificielle ne peut pas être ajoutée seule à un processus de cybersécurité. Elle doit être entourée de validations déterministes, d'une traçabilité forte, d'une mémoire structurée, de règles de sécurité et d'un contrôle humain. Le projet illustre cette approche en séparant la génération IA de la décision finale.

Le travail reste néanmoins un prototype académique. Pour une utilisation en production, il faudrait renforcer l'authentification, ajouter une observabilité complète, automatiser les tests frontend, valider le mode IA réel dans des conditions stables, élargir la couverture ATT&CK et connecter la publication à un SIEM réel. Ces perspectives constituent une continuité naturelle du travail réalisé.

Bibliographie

- [ADRDS⁺20] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Benmetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (xai) : Concepts, taxonomies, opportunities and challenges toward responsible ai. *Information Fusion*, 58 :82–115, 2020.
- [For26a] Forvis Mazars Group. About us. <https://www.forvismazars.com/group/en/who-we-are/about-us>, 2026.
- [For26b] Forvis Mazars Group. Brand identity. <https://www.forvismazars.com/>, 2026.
- [For26c] Forvis Mazars Group. Cyber security consulting. <https://www.forvismazars.com/group/en/services/consulting/technology-digital-consulting/cyber-security-consulting>, 2026.
- [MIS26] MISP Project. Misp - open source threat intelligence and sharing platform. <https://www.misp-project.org/>, 2026. Consulté pour le contexte technique du projet.
- [MIT26a] MITRE. Mitre att&ck enterprise matrix. <https://attack.mitre.org/>, 2026.
- [MIT26b] MITRE. Mitre d3fend knowledge graph. <https://d3fend.mitre.org/>, 2026.
- [Nat24] National Institute of Standards and Technology. The nist cybersecurity framework 2.0. <https://www.nist.gov/cyberframework>, 2024.
- [Sig26a] SigmaHQ. pysigma documentation. <https://sigmahq-pysigma.readthedocs.io/>, 2026.
- [Sig26b] SigmaHQ. Sigma - generic signature format for siem systems. <https://sigmahq.io/>, 2026.

Annexe A

Annexe A : Organisation technique

L'organisation technique de la solution repose sur les ensembles suivants :

- **backend/** : application FastAPI, modèles, services, routes, workers Celery et migrations Alembic ;
- **frontend/** : application Next.js, pages SOC, composants partagés et client API ;
- **docs/** : documentation technique, architecture, intégration MISP, API, tests, diagrammes et audits ;
- **scripts/backend/** : tests et scénarios de vérification backend ;
- **docker/** : configuration Nginx ;
- **artifacts/screenshots/** : captures d'écran de l'interface.

Services Docker

Les services principaux sont **backend**, **frontend**, **nginx**, **postgres**, **redis**, **worker** et **scheduler**. Le profil **misp** ajoute MISP et ses dépendances. L'interface de la plateforme est accessible localement sur **http://localhost:8080**. L'interface MISP locale est documentée sur **https://localhost:8443**.

Annexe B

Annexe B : Exemples de configuration

Variables MISP

```
CTI_MISP_URL=https://misp
CTI_MISP_API_KEY=cle_api_misp_generee_depuis_interface
CTI_MISP_VERIFY_TLS=false
```

La clé API doit être créée depuis l'interface MISP avec un utilisateur autorisé à utiliser l'API. Les tests d'ingestion doivent passer par l'API MISP réelle et non par une insertion directe dans PostgreSQL.

Commande de démarrage local

```
docker compose --profile misp up -d --build
```

Exécution des tests backend

```
make backend-test
make backend-lint
make compose-config
```

Ces commandes nécessitent un environnement Docker fonctionnel pour les vérifications complètes.